

Standard Input in C++ Using `cin`

1. What is Standard Input?

In C++, **standard input** is used to take data from the user through the keyboard.

The standard input object in C++ is:

```
cin
```

`cin` is provided by the `<iostream>` library.

We commonly use `cin` to take:

- Integers
- Floating-point numbers
- Characters
- Words
- Multiple values

2. Basic Syntax of `cin`

The basic syntax is:

```
cin >> variable;
```

Understanding the Syntax

- `cin` → standard input object.
- `>>` → **extraction operator**, which takes data from the keyboard.
- `variable` → the variable where the input is stored.

Think of it as:

```
Keyboard → cin → Variable
```

Example

```
int age;  
  
cin >> age;
```

If the user enters:

```
20
```

the value `20` is stored in the variable `age`.

3. Taking Multiple Inputs

We can take multiple values using a single `cin` statement.

Syntax

```
cin >> a >> b >> c;
```

For example:

```
int a, b, c;  
  
cin >> a >> b >> c;
```

If the user enters:

```
10 20 30
```

then:

```
a = 10  
b = 20  
c = 30
```

Important

The user can enter the values separated by **spaces** or by pressing **Enter**.

For example, both are valid:

```
10 20 30
```

or:

```
10  
20
```

30

4. Input an Integer

Question

Write a C++ program that asks the user to enter an integer and displays the entered value.

Program

```
#include <iostream>
using namespace std;

int main() {

    int num;

    cout << "Enter an integer: ";
    cin >> num;

    cout << "You entered: " << num;

    return 0;
}
```

Sample Output

```
Enter an integer: 25
You entered: 25
```

Understand the Code

```
int num;
```

Creates an integer variable named `num`.

```
cin >> num;
```

Takes an integer from the user and stores it in `num`.

```
cout << "You entered: " << num;
```

Displays the value stored in `num`.

5. Input Two Numbers and Add Them

Question

Write a C++ program that takes two integers from the user and displays their sum.

Program

```
#include <iostream>
using namespace std;

int main() {

    int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;

    cout << "Sum = " << a + b;

    return 0;
}
```

Sample Output

```
Enter two numbers: 10 20
Sum = 30
```

Understand the Code

```
cin >> a >> b;
```

takes two values from the user:

- First value → stored in **a**
- Second value → stored in **b**

Then:

```
a + b
```

calculates their sum.

6. Input a Floating-Point Number

Question

Write a C++ program that reads a floating-point number from the user and displays it.

Program

```
#include <iostream>
using namespace std;

int main() {

    float value;

    cout << "Enter a floating-point value: ";
    cin >> value;

    cout << "You entered: " << value;

    return 0;
}
```

Sample Output

```
Enter a floating-point value: 12.5
You entered: 12.5
```

Remember

Use `float` when you want to store numbers that may contain a decimal part.

For example:

```
12.5
7.25
100.75
```

7. Input a Character

Question

Write a C++ program that asks the user to enter a single character and displays it.

Program

```
#include <iostream>
using namespace std;

int main() {

    char ch;

    cout << "Enter a character: ";
    cin >> ch;

    cout << "You entered: " << ch;

    return 0;
}
```

Sample Output

```
Enter a character: A
You entered: A
```

Remember

A **char** variable stores a **single character**.

Examples:

```
A
B
7
$
```

8. Input a Word

Question

Write a C++ program that reads a single word from the user and displays a welcome message.

Program

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    string name;
```

```
cout << "Enter your name: ";  
cin >> name;  
  
cout << "Welcome, " << name << "!";  
  
return 0;  
}
```

Sample Output

```
Enter your name: Ali  
Welcome, Ali!
```

Understand the Code

```
string name;
```

creates a variable named **name** that can store text.

```
cin >> name;
```

takes a **single word** from the user.

Important

cin >> stops reading when it encounters a **space**.

For example, if the user enters:

```
Muhammad Yasir
```

with:

```
cin >> name;
```

only:

```
Muhammad
```

will be stored in **name**.

To read a complete line containing spaces, we use **getline()**:

```
getline(cin, name);
```

We will study `getline()` separately.

9. `cin` with Different Data Types

The type of variable determines what kind of input we expect.

Data Type	Example Input	Example Variable
<code>int</code>	25	<code>int age;</code>
<code>float</code>	12.5	<code>float marks;</code>
<code>double</code>	125.75	<code>double price;</code>
<code>char</code>	A	<code>char grade;</code>
<code>string</code>	Ali	<code>string name;</code>

Example:

```
int age;
float marks;
char grade;
string name;

cin >> age;
cin >> marks;
cin >> grade;
cin >> name;
```

10. `cin` and `cout` Together

In most beginner programs, `cin` and `cout` are used together.

- `cout` → displays a message or result.
- `cin` → takes input from the user.

Example

```
#include <iostream>
using namespace std;

int main() {
```

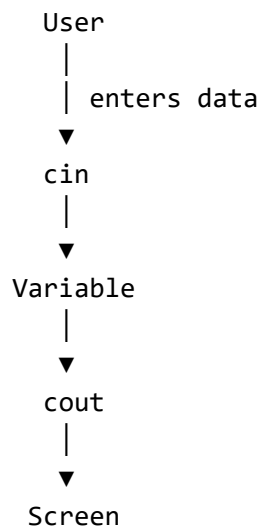
```
int age;

cout << "Enter your age: ";
cin >> age;

cout << "Your age is: " << age;

return 0;
}
```

Think of the process as:



11. `std::cin`

We can also use `cin` without:

```
using namespace std;
```

In that case, we write:

```
std::cin >> age;
```

Example

```
#include <iostream>

int main() {
    int age;
```

```
std::cout << "Enter your age: ";
std::cin >> age;

std::cout << "Your age is: " << age;

return 0;
}
```

Here, `std::cin` means that `cin` belongs to the **standard (std) namespace**.

Quick Review

Component	Purpose
<code>#include <iostream></code>	Provides input/output features
<code>cin</code>	Takes input from the user
<code>>></code>	Extraction operator
<code>cout</code>	Displays output on the screen
<code><<</code>	Insertion operator
<code>int</code>	Stores integer values
<code>float</code>	Stores decimal values
<code>char</code>	Stores a single character
<code>string</code>	Stores text

Key Points to Remember

`cin`

Used to **take input from the user**.

```
cin >> age;
```

`>>`

Called the **extraction operator**.

```
cin >> variable;
```

It extracts data from the input stream and stores it in the variable.

Multiple Inputs

```
cin >> a >> b >> c;
```

cout vs cin

```
cin  → Input  → User → Program  
cout → Output → Program → Screen
```

Practice Questions

Practice 1 – Student Age

Write a C++ program that asks the user to enter their age and displays:

```
Your age is: 20
```

Practice 2 – Two Numbers

Write a C++ program that takes two integers from the user and displays their:

- Sum
- Difference
- Product

Practice 3 – Student Marks

Write a C++ program that takes marks of three subjects and displays the total marks.

Practice 4 – Rectangle

Write a C++ program that takes the length and width of a rectangle and calculates its area.

Formula:

```
Area = Length × Width
```

Practice 5 – Student Information

Write a C++ program that takes the following information from the user:

- Name
- Age
- Department

- Marks

Then display all the information using `cout`.

Remember

```
cin  → Takes input from the user
cout → Displays output on the screen

>>  → Extraction operator
<<  → Insertion operator
```

`cin` is used for input, while `cout` is used for output.